Universidad Tecnológica del Perú

DISEÑO E IMPLEMENTACIÓN DE UN

SISTEMA DE BANCA MÓVIL



INTEGRANTES:

RAMIREZ FLORES, GIOMARA
COELLO FLORES, MILAGROS
CUENCA HUARACA, PIERINA
GAMBOA MUÑOZ, JESUS
VEGA QUISPE, MATIAS

Expone: Gio

INTRODUCCIÓN



- La transformación digital cambió la forma de relacionarse con las entidades financieras.
- Las aplicaciones permiten consultar saldos y movimientos, transferir dinero, pagar servicios y gestionar productos.
- El proyecto propone centralizar operaciones en un sistema organizado y seguro.
- Considera cinco roles: Administrador, Cliente, Asesor de Crédito, Cajero y Auditor.

Expone: Gio

Definición del problema

- Los procesos presenciales o los sistemas separados pueden generar demoras y duplicidad de tareas.
- También dificultan la consulta oportuna de información y la experiencia del usuario.



- Se identifica la necesidad de integrar las operaciones según el rol de cada usuario.
- La propuesta es académica: no se conectará con bancos ni procesará dinero real.

Expone: Gio

Funciones del Sistema

Cada actor tiene su propio panel y solo ve sus opciones.

01 **Cliente:** consulta saldos y movimientos, transfiere, paga servicios y solicita créditos.

02 **Administrador:** gestiona usuarios y roles, y configura los parámetros del banco.

03 **Asesor:** aprueba o rechaza solicitudes de crédito con observación y consulta el historial del cliente.

04 **Cajero:** registra clientes y abre cuentas; registra depósitos y retiros con comprobante.

05 **Auditor:** revisa todas las transacciones y consulta la bitácora de accesos.

Expone: Gio

Objetivo General

Diseñar y desarrollar un sistema de banca digital que permita gestionar de forma integrada las principales operaciones de una entidad financiera, considerando los roles de sus usuarios y aplicando principios SOLID, patrones de diseño y una arquitectura por capas para obtener un sistema organizado, mantenible y escalable.

Objetivos Específicos

Implementar un mecanismo de autenticación y autorización que permita controlar el acceso según el rol del usuario.

Organizar la solución mediante una arquitectura por capas que separe la presentación, la lógica de negocio y el acceso a datos.

Aplicar principios SOLID y patrones de diseño que faciliten el mantenimiento, la reutilización y la extensión del sistema.

Desarrollar las funcionalidades principales del cliente, como consulta de saldos y movimientos, transferencias, solicitudes de crédito y pago de servicios.

Desarrollar funciones administrativas y operativas para la gestión de usuarios, créditos, depósitos, retiros, comprobantes y auditoría.

Utilizar una base de datos relacional para almacenar de manera estructurada la información de clientes, cuentas, operaciones, créditos y usuarios.

Expone: Milagros

Alcances

- Gestión de usuarios y roles.
- Consulta de saldos y movimientos.
- Registro de transferencias y pagos dentro del proyecto.
- Registro y evaluación de solicitudes de crédito.
- Registro de depósitos, retiros y emisión de comprobantes.
- Consulta de operaciones y accesos para auditoría.

Limitaciones

- No habrá conexión con cuentas bancarias reales.
- No se realizarán transferencias ni pagos con dinero real.
- No se contempla el despliegue en producción.
- La seguridad será de nivel académico, no equivalente a la bancaria real.
- Quedan fuera las integraciones con centrales de riesgo, mensajería y proveedores externos de identidad.

Expone: Milagros

MARCO TEÓRICO

Search...

Antecedentes

NACIONALES

- **BCP – Banca Móvil (2024):** transferencias, pagos, saldos y movimientos con mecanismos de seguridad.
- **Interbank APP:** centralización de operaciones, consultas, transferencias y pagos.
- **BBVA Perú – App/Cuenta Digital (2025):** productos digitales y transferencias inmediatas.
- **Aporte:** orientar los módulos de cuentas, pagos, transferencias y autenticación.

INTERNACIONALES

- **Bank of America:** referencia para escalabilidad, seguridad y disponibilidad.
- **Revolut:** servicios bancarios, ahorro y crédito desde una plataforma.
- **Nubank:** pagos móviles, límites y pagos programados.
- **Aporte:** considerar una plataforma integral y operaciones digitales organizadas.

Expone: Pierina

Search...

FUNDAMENTO TEÓRICO





El fundamento teórico reúne los conceptos que sirven de base para el diseño del sistema de banca móvil, buscando una estructura organizada, mantenible y escalable.

Conceptos principales:

Banca digital.
Principios SOLID.
Patrones de diseño de software.
Patrones GRASP.
Arquitectura en capas.

Expone: Pierina

Estructura del Proyecto

banca-digital/	
├─ pom.xml	Spring Boot, Java 21, PostgreSQL
├─ sql/schema.sql	crea las 7 tablas
├─ src/main/resources/	
│ ├─ application.properties	puerto y conexión a PostgreSQL
│ └─ templates/	VISTAS Thymeleaf, una carpeta por actor
└─ src/main/java/com/banco/	
├─ BancaDigitalApplication	punto de arranque
├─ config/	seguridad y datos iniciales
├─ controller/	CONTROLADORES (uno por actor)
├─ service/	REGLAS DE NEGOCIO
├─ patterns/adapter/	PATRÓN ADAPTER (pago de servicios)
├─ dao/ · impl/ · factory/	ACCESO A DATOS con JDBC (Factory)
├─ model/	ENTIDADES
└─ util/	Singleton, transacciones, validaciones

Diseño de la página:
templates/

Reglas del negocio:
controller/ · service/
model/ · patterns/

Base de datos:
dao/ · util/
sql/schema.sql

Seguridad:
config/SecurityConfig
UsuarioDetailsService

Expone: Matías

¿Qué hemos usado?

Herramienta	Para qué
Java 21	Lenguaje del proyecto
Spring Boot 3.3.5	Servidor web y rutas; sin JPA ni Hibernate
Maven	Dependencias y compilación
Spring Security + BCrypt	Login, roles, CSRF y claves cifradas
Thymeleaf + Bootstrap 5.3.3	Vistas HTML dinámicas y responsivas
PostgreSQL + pgAdmin	Base de datos relacional y su administrador
JDBC + PreparedStatement	Consultas seguras (anti inyección SQL) y transacciones
IntelliJ IDEA	Entorno de desarrollo

Expone: Matías

Arquitectura y Capa de Presentación

Es la interfaz visual y pasarela a la lógica de negocio.

- Tecnologías: Spring Boot 3.3.5, Java 21 (MVC), Thymeleaf y Bootstrap 5.3.3 (local vía Maven).
- Seguridad visual: el módulo de seguridad de Thymeleaf muestra u oculta elementos según el rol.

Estructura del Sistema

El sistema tiene cinco actores: cliente, cajero, asesor de crédito, auditor y administrador. Cada uno cuenta con su propio controlador y sus páginas. Además, hay cuatro páginas comunes: inicio de sesión, fragmentos reutilizables y las páginas de error. En total, el sistema tiene 25 plantillas.

Funcionamiento de los Controladores (GRASP)

- Delegación: procesan peticiones y devuelven vistas sin lógica de negocio.
- Patrón PRG: evita duplicar transacciones al recargar, con mensajes flash.
- Seguridad: usa la sesión autenticada para evitar accesos indebidos.
- Errores: gestiona excepciones de negocio y oculta detalles de la base de datos.

Thymeleaf y Reutilización

- Fragmentos: plantillas reutilizables para cabecera, navegación, alertas y formato de dinero.
- Control de acceso: sec:authorize para menús por rol y sec:authentication para el usuario.
- Seguridad en formularios: protección automática contra CSRF.

Expone: Matías

Arquitectura en 3 capas

Navegador → **SecurityConfig**: ¿está logueado? ¿su rol puede entrar?

Presentación
View

templates/ HTML + Thymeleaf + Bootstrap · **controller/** un controlador por actor (GRASP Controlador): recibe la petición y delega

Lógica de negocio
Service

service/ reglas y transacciones · **model/** Cuenta valida su saldo (GRASP Experto) · **patterns/adapter/** pago de servicios

Acceso a datos
DAO

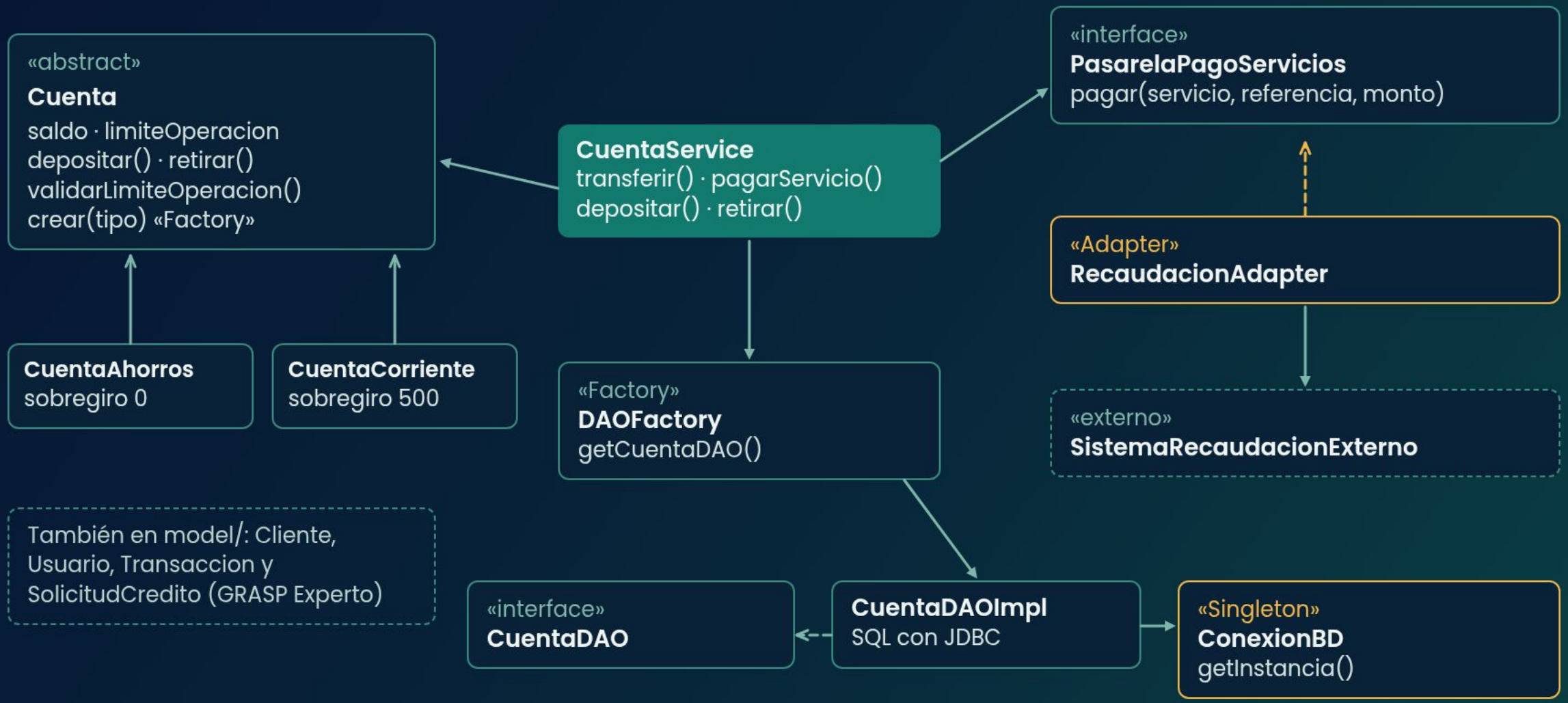
dao/ interfaces · **dao/impl/** SQL con JDBC · **DAOFactory** (Factory) · **ConexionBD** (Singleton)

PostgreSQL · 7 tablas

Expone: Gio

Diagrama de clases (núcleo)

—▷ hereda · - - ▷ implementa · → usa



Expone: Matías

Patrones y principios aplicados

Categoría	Patrón	Clase	Para qué
Creacional	Singleton	ConexionBD	Una única conexión a la base de datos
Creacional	Factory	DAOFactory · Cuenta.crear()	Crear DAOs y tipos de cuenta sin acoplarse a clases concretas
Estructural	Adapter	RecaudacionAdapter	Usar un sistema externo de pagos con otra interfaz
GRASP	Controlador	Un controller por actor	Recibe la petición y delega en los servicios
GRASP	Experto	Cuenta · SolicitudCredito	Quien tiene los datos hace el cálculo
SOLID	O · L	CuentaAhorros · CuentaCorriente	Nuevos tipos de cuenta sin tocar Cuenta
SOLID	D	Interfaces DAO · PasarelaPagoServicios	Los servicios dependen de abstracciones

Expone: Pierina

Base de datos: 7 tablas en PostgreSQL

cliente
id · nombre
dni (UNIQUE) · correo
telefono

usuario
username (UNIQUE)
password_hash (BCrypt)
rol · activo · cliente_id

cuenta
numero (BD-...) · tipo
saldo · limite_operacion
cliente_id

transaccion
cuenta_id · tipo
monto (CHECK > 0)
saldo_resultante · fecha

solicitud_credito
cliente_id · monto · plazo
estado · observaciones
asesor_id

bitacora_acceso
username · evento
(LOGIN_OK, LOGIN_FALLO, LOGOUT)
detalle · fecha

parametro
LIMITE_OPERACION_DEFECTO
MONTO_MAXIMO_CREDITO

Relaciones
cliente 1 — 1 usuario
cliente 1 — * cuenta
cuenta 1 — * transaccion
cliente 1 — * solicitud

Las restricciones (UNIQUE, CHECK, FOREIGN KEY) protegen los datos aunque fallara Java. Los parámetros evitan números mágicos en el código.

Expone: Milagros

Seguridad: 3 pilares



Autenticación

¿Quién eres?

Spring Security + clave cifrada con BCrypt



Autorización

¿Qué puedes hacer?

5 roles; cada uno solo entra a sus rutas



Auditoría

¿Qué hiciste?

Bitácora de ingresos, salidas e intentos fallidos

Expone: Jesús

Capas de seguridad

BCrypt
Claves guardadas como hash, nunca en texto
`SecurityConfig · UsuarioService`

Control por rol
Cada actor solo entra a sus rutas
`SecurityConfig`

Token CSRF
Formularios con token anti-falsificación
`Spring Security + Thymeleaf`

PreparedStatement
Consultas parametrizadas contra inyección SQL
`dao/impl/*`

Bitácora de accesos
Ingresos, salidas e intentos fallidos
`AuditoriaService`

Usuario desactivable
El admin bloquea una cuenta sin borrarla
`UsuarioDetailsService`

Validaciones
DNI, usuario, clave y montos válidos
`util/Validaciones`

Errores genéricos
Sin detalles técnicos hacia el usuario
`ManejadorErrores`

Expone: Jesús

¿Cómo se ejecuta de manera interna?

Cuando el cajero registra un cliente, se piden su DNI, una contraseña y datos personales como correo y teléfono. El cliente y su usuario de acceso (usuario = DNI) se guardan juntos en la base de datos, en una sola transacción.

Pero ¿cuál es el flujo internamente en el programa?



Expone: Jesús

Lo que viene en el siguiente avance

Estructural

Facade

Una sola llamada para operaciones que combinan varios servicios.

Comportamiento

Observer

Notificar al cliente cada vez que se mueve su cuenta.

Comportamiento

State

Estados del crédito (pendiente, aprobado, rechazado) sin if encadenados.

Nueva función

Cuenta de crédito

Al aprobarse un crédito se crea una cuenta para pagar y transferir (CuentaCredito, vía Factory).

SOLID

Segregación (I)

Separar interfaces de lectura y escritura en los DAO.

Actores

Más funciones

Llegar a 4 funciones por actor, como pide la entrega final.

Expone: Pierina

CONCLUSIONES

Las plataformas tienden a centralizar operaciones financieras en un mismo canal.

La autenticación y el control de acceso son elementos importantes.

La facilidad de uso y la disponibilidad forman parte de la experiencia digital.

Los antecedentes orientan el diseño; el proyecto académico no reproduce exactamente esas plataformas.

Singleton, Factory y Adapter, junto con la arquitectura en capas, dieron un código ordenado y fácil de extender.

Las transacciones JDBC garantizan que el dinero nunca quede a medias: todo se guarda o nada.

Expone: Jesús